



Lab Sheet -05

Data Structure I (BTCS209P)

B.Tech III Semester (Session 2026-2027)

Objective:

To understand the concept of Stack, its LIFO (Last In, First Out) principle, basic operations, array representation, algorithms, and applications using C++.

Requirements:

C++ Compiler / IDE

VS Code, Code::Blocks, Dev-C++, or any suitable C++ environment

Basic knowledge of arrays, loops, functions, and conditional statements

Concepts Covered:

1. Stack and LIFO principle
2. Stack representation using arrays
3. TOP variable
4. Push operation
5. Pop operation
6. Peek operation
7. Display operation
8. Stack Overflow and Stack Underflow
9. Applications of Stack

1. Stack

A Stack is a linear data structure in which insertion and deletion are performed from only one end, called the TOP. Stack follows the LIFO (Last In, First Out) principle.

Example:

Push: 10 → 20 → 30

TOP

↓

[30] ← Last inserted

[20]

[10]

Pop() removes 30 first.

2. Basic Stack Operations

Push

Push inserts a new element at the TOP of the stack. If the stack is full, Stack Overflow occurs.



Pop

Pop removes and returns the element present at the TOP. If the stack is empty, Stack Underflow occurs.

Peek

Peek returns the TOP element without removing it.

Display

Display prints the stack elements from TOP to the bottom.

3. Stack Representation Using Array

In an array implementation, a variable TOP keeps track of the current top element.

Initially:

TOP = -1

Empty Stack:

TOP == -1

Full Stack:

TOP == MAX - 1

4. Stack Overflow and Underflow

Stack Overflow occurs when we try to insert an element into a full stack. Stack Underflow occurs when we try to delete an element from an empty stack.

Overflow → TOP == MAX - 1

Underflow → TOP == -1

5. Algorithms of Stack Operations

Push Algorithm

PUSH(STACK, ITEM)

1. If TOP == MAX - 1
2. Print "Stack Overflow"
3. Stop
4. TOP = TOP + 1
5. STACK[TOP] = ITEM
6. Stop

Pop Algorithm

POP(STACK)

1. If TOP == -1
2. Print "Stack Underflow"
3. Stop



4. ITEM = STACK[TOP]
5. TOP = TOP - 1
6. Return ITEM
7. Stop

Peek Algorithm

PEEK(STACK)

1. If TOP == -1
2. Print "Stack is Empty"
3. Stop
4. Return STACK[TOP]
5. Stop

Display Algorithm

DISPLAY(STACK)

1. If TOP == -1
2. Print "Stack is Empty"
3. Stop
4. For i = TOP down to 0
5. Print STACK[i]
6. Stop

6. C++ Implementation Using Array

```
#include <iostream>
using namespace std;

#define MAX 5

int stackArr[MAX];
int top = -1;

void push(int value)
{
    if (top == MAX - 1)
    {
        cout << "Stack Overflow\n";
        return;
    }
    top++;
    stackArr[top] = value;
}

void pop()
{
    if (top == -1)
    {
        cout << "Stack Underflow\n";
    }
}
```



```
return;

}
cout << stackArr[top] << " popped\n";
top--;
}

void peek()
{
    if (top == -1)
    {
        cout << "Stack is Empty\n";
        return;
    }
    cout << "Top element = " << stackArr[top] << "\n";
}

void display()
{
    if (top == -1)
    {
        cout << "Stack is Empty\n";
        return;
    }

    for (int i = top; i >= 0; i--)
        cout << stackArr[i] << " ";

    cout << endl;
}

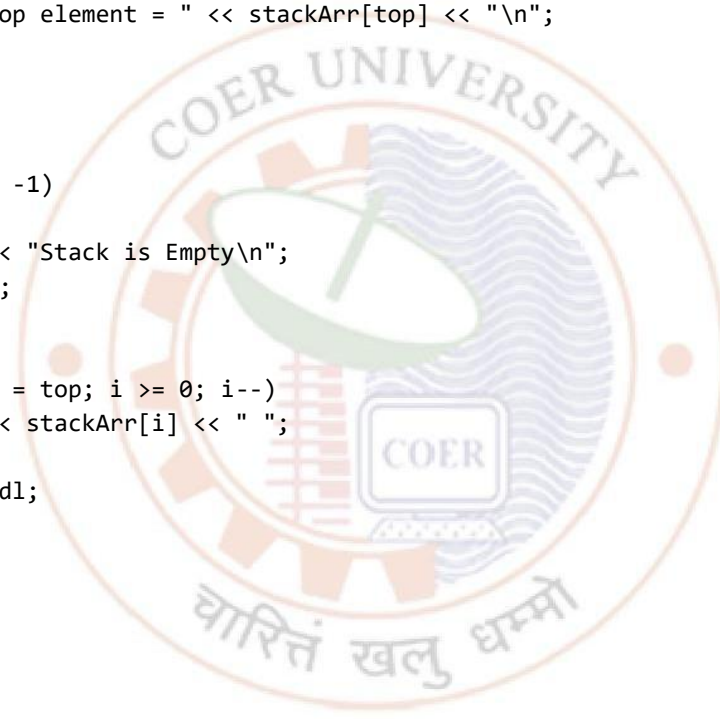
int main()
{
    push(10);
    push(20);
    push(30);

    display();
    peek();
    pop();
    display();

    return 0;
}
```

7. Time Complexity of Stack Operations

Push → $O(1)$
Pop → $O(1)$
Peek → $O(1)$
Display → $O(n)$



8. Applications of Stack

1. Function calls and recursion
2. Infix, Prefix, and Postfix expression conversion
3. Expression evaluation
4. Parentheses matching
5. Undo/Redo operations
6. Backtracking
7. Browser history
8. Depth First Search (DFS)

9. Stack for Parentheses Matching

A Stack can be used to check whether brackets in an expression are balanced. Opening brackets are pushed and matching opening brackets are removed when closing brackets are encountered.

Example:

{ [()] }

{ → Push
[→ Push
(→ Push
) → Pop
] → Pop
} → Pop

Stack becomes empty → Balanced

Lab Exercises

1. Write a C++ program to implement a Stack using an array with Push, Pop, Peek, and Display operations.
2. Write a C++ program to demonstrate Stack Overflow.
3. Write a C++ program to demonstrate Stack Underflow.
4. Write a C++ program to display the TOP element using Peek operation.
5. Write a menu-driven C++ program for Push, Pop, Peek, and Display.
6. Write a C++ program to implement a Stack of integers using an array.
7. Write a C++ program to reverse a string using Stack.
8. Write a C++ program to check balanced parentheses using Stack.
9. Write a C++ program to convert an Infix expression into Postfix using Stack.
10. Write a C++ program to evaluate a Postfix expression using Stack.
11. Write a C++ program to convert an Infix expression into Prefix using Stack.
12. Write a C++ program to implement two stacks in a single array.
13. Write a C++ program to implement Stack using a class.
14. Write a C++ program to demonstrate Stack use in recursion.
15. Write a C++ program to analyze the time complexity of Push, Pop, Peek, and Display operations.



Instructions & Guidelines

- Write a separate C++ program for each assigned exercise.
- Use functions wherever appropriate to make programs modular.
- Do not use the STL stack for basic array implementation exercises unless specifically instructed.
- Clearly handle Stack Overflow and Stack Underflow conditions.
- Test programs with empty, partially filled, and full-stack cases.
- Add comments to important parts of the program.
- Mention time and space complexity of implemented operations.

Submission

Submit the C++ source code files for the assigned tasks along with a brief report containing the algorithm, sample input/output, and time and space complexity analysis.

